



Stored Procedure

A **Stored Procedure** is a saved collection of SQL statements stored inside the database and executed whenever needed using the CALL command.

Stored procedures help automate repetitive tasks, improve performance, centralize business logic, and make SQL applications easier to maintain.

Short Notes and Explanation of Each Script

SECTION 1: Basic Query

```
SELECT *  
FROM job_roles  
WHERE base_salary >= 100000;
```

What It Does

- Retrieves all job roles with salaries greater than or equal to 100000.
- Demonstrates a normal SQL filtering query before converting it into a stored procedure.

Key Concept

- Uses WHERE clause for filtering records.

SECTION 2: Simple Stored Procedure

```
CREATE PROCEDURE high_salary_roles()  
SELECT *  
FROM job_roles  
WHERE base_salary >= 100000;
```

What It Does

- Creates a stored procedure named high_salary_roles.
- Saves the query permanently inside the database.

Key Concept

- Stored procedures allow reusable SQL logic.



SECTION 3: Executing Stored Procedure

```
CALL high_salary_roles();
```

What It Does

- Executes the stored procedure.
- Returns all high-paying job roles.

Key Concept

- CALL is used to run stored procedures.

SECTION 4: Stored Procedure with Multiple Queries

```
DELIMITER $$

CREATE PROCEDURE salary_reports()
BEGIN

    SELECT *
    FROM job_roles
    WHERE base_salary >= 100000;

    SELECT *
    FROM job_roles
    WHERE base_salary >= 70000;

END $$

DELIMITER ;
```

Key Concepts

- DELIMITER \$\$ changes the default statement separator.
- BEGIN ... END groups multiple SQL statements together.



SECTION 5: Execute Multi-Query Procedure

```
CALL salary_reports();
```

What It Does

- Runs the multi-query stored procedure.
- Produces multiple result sets.

Key Concept

- One procedure can return several outputs.

SECTION 6: Drop Procedure If Exists

```
DROP PROCEDURE IF EXISTS employee_salary_filter;
```

What It Does

- Deletes the stored procedure if it already exists.

Why It Is Important

- Prevents duplicate procedure creation errors.

Key Concept

- Useful during development and updates.

SECTION 7: Stored Procedure with Parameters

```
CREATE PROCEDURE employee_salary_filter(employee_id_param INT)
```

What It Does

- Creates a procedure that accepts user input dynamically.
- Filters employee data using employee ID.



Query Purpose

Returns:

- Employee details
- Department
- Job role
- Salary information

Key Concept

- Parameters make procedures dynamic and reusable.

SECTION 8: Executing Parameterized Procedure

```
CALL employee_salary_filter(1);
```

What It Does

- Runs the procedure using parameter value 1.
- Retrieves salary information for employee ID 1.

Key Concept

- Input values can be passed during execution.

SECTION 9: Advanced Business Analytics Procedure

```
SELECT *  
FROM employee_salary_analysis  
ORDER BY base_salary DESC;
```

What It Does

Displays employees ordered from highest salary to lowest.

Documentation

Useful for payroll analysis and management reporting.



SECTION 8: BUSINESS ANALYTICS EXAMPLE

```
CREATE PROCEDURE department_salary_analysis()
```

What It Does

Generates department salary analytics including:

- Total employees
- Average salary
- Highest salary
- Lowest salary
- Total payroll cost

Business Purpose

Useful for:

- HR reporting
- Payroll analysis
- Financial dashboards
- Budget planning

Key Concepts

Uses:

- COUNT()
- AVG()
- MAX()
- MIN()
- SUM()
- GROUP BY
- ORDER BY

SECTION 10: Execute Analytics Procedure

```
CALL department_salary_analysis();
```

What It Does

- Runs the department analytics procedure.
- Displays payroll insights for all departments.



Key Concept

- Stored procedures simplify complex reporting.

Additional Documentation

Why Stored Procedures Are Important

Stored Procedures help:

- Reduce repeated SQL code
- Improve database performance
- Secure sensitive queries
- Standardize business logic
- Simplify enterprise applications

Important SQL Keywords Used

Keyword	Purpose
CREATE PROCEDURE	Creates stored procedure
CALL	Executes procedure
DELIMITER	Changes statement separator
BEGIN ... END	Defines procedure block
DROP PROCEDURE	Removes procedure
PARAMETERS	Accepts dynamic input
GROUP BY	Groups records
ORDER BY	Sorts results



Real-World Use Cases

Stored Procedures are heavily used in:

- Payroll systems
- HR management systems
- Banking applications
- ERP systems
- Financial reporting
- Data analytics dashboards
- Enterprise automation systems